Applied Software Development Lab

Lab Handout 4: Fun and games with the serial interface

Learning objectives

- Understanding serial communication between the microcontroller and the computer.
- Applying the basics of the Python programming language.
- A little bit of `git`

Step 1: Forking the repository

Goal: From now on, we will develop a software project step by step. We will proceed one step at a time – and things will surely go wrong now and then. But that is exactly what we have git for – a tool for version control.

1. Open the following URL in your web browser: http://gitlab.uni-ulm.de/mlehn/asp-ulm
2. Log in with your kiz account.
3. Click **Fork** at the top right.
4. Under **Select a namespace**, choose your kiz username. Then click **Fork project** at the bottom.

5. You are now on the project page of your own fork. Click **Code** there and copy the link under **Clone with HTTPS**.
   The link has the form: https://gitlab.uni-ulm.de/**USERNAME**/asp-ulm
6. As usual, we work with two terminals side by side – the left one for editing, the right one for running commands.

Left terminal (editing)

A) **Clone your Git repository**
   Clone your fork with:
   `git clone https://gitlab.uni-ulm.de/USERNAME/asp-ulm`
   Then change into the directory with
   `cd asp-ulm`

B) **Change a file as a test**
   Open the file `README.md` and write in a new line: `Here we go!`

Right terminal (running commands)

A) Also change into the project directory with `cd asp-ulm`
   Now try to commit the change with `git commit -a`
   At first, an error message appears, something like: **"Please tell me who you are…"**

B) Then run the following two commands (with your own name and email address):
   `git config user.email "your.name@uni-ulm.de"`
   `git config user.name "Your Name"`

C) Now run the commit again:
   `git commit -a`
   nano opens. Enter a short message, e.g.: `My first commit`
   Exit nano with Control + X and confirm with y.

D) Transfer your commit to the server:
   `git push`
   You will be asked for your kiz username and password.
   Note: **the password is not displayed while you type**, not even as asterisks – that is normal. Just type it blind and press Enter.

E) Switch back to the browser, reload the page, and check whether your change has been applied.

Step 2: Programming the microcontroller with the Arduino framework

Preparation: In both terminals, change into the subdirectory `controller`.

Left terminal: In the left terminal, open the file `controller.ino` with `nano`. It contains the following program:

```
 1 void setup()
 2 {
 3     Serial.begin(9600);
 4     Serial.println("Hello! Lots of A's coming up");
 5 }
 6
 7 void loop()
 8 {
 9     Serial.print("A");
10     delay(1000);
11 }
```

Right terminal

1. **Connect the microcontroller.**
   Attention: under WSL, you have to run the command `usb_check` every time you connect a microcontroller so that the USB port becomes visible in WSL.
2. **Compile and upload the program:** `make upload`
3. **Start the serial monitor:** `make monitor`
   First, a line should appear with: `Hello! Lots of A's coming up` and then – as promised – lots of `A`s. :-)
   You can exit the serial monitor with `Control-X`.

Brief explanation of what happens
`controller.ino` actually contains only a small excerpt of the complete C program that is compiled in the end. The functions `setup()` and `loop()` are – as shown here – embedded internally in a `main()` function:

```
// Simplified representation – in reality, there are many more includes
// and initializations here

int main()
{
    // setup() is called exactly once
    setup();

    // After that, loop() is called again and again
    while (1) {
        // With Arduino, an internal function checks, e.g., whether serial
        // data is available:
        if (Serial.available()) { /* ... */}

        loop();
    }
}
```

So the function `setup()` is called exactly once – in our example, to initialize the serial interface and to output a line of text.
The function `loop()`, on the other hand, runs in an endless loop: in each pass, the character `A` is output, followed by a wait of 1000 milliseconds (i.e., one second).

© Michael Lehn | CC BY-SA 4.0

What you should remember for today:

- With `Serial.begin(9600)`, you initialize the serial interface – here with a baud rate of 9600.
- With `Serial.print("foo")`, the string `"foo"` is output without an additional line break.
- With `Serial.println("foo")`, the string `"foo"` is output followed by a line break (more precisely: `\r\n`).

Open questions: How exactly does the serial interface work?

When `Serial.print("A");` is executed, you can observe on pin 1 (TX) with the oscilloscope how the character `'A'` is transmitted – as a sequence of voltage changes that correspond to the bit pattern of `0x41`.
The letter `'A'` has a so-called ASCII value and is represented as an 8-bit number: `01000001`. These bits are sent one after the other – similar to Morse code – by setting the TX pin **HIGH** or **LOW**. At a baud rate of **9600**, each bit takes about 104 µs.

But: The data is not only output via **pin 1 (TX)**, but is **also transmitted over the USB cable** – with the help of a **serial-to-USB converter** (often integrated into the microcontroller or present on the board). That is why you can also see the transmitted character in the **serial monitor on the computer**, without any oscilloscope.

Finally

Change line 4 so that only `Hello!` followed by a line break is output. Commit the changes and upload them to the repository with the following command:

```
git commit -a -m "Step 2"
git push
```

Step 3: Playing around with the Arduino framework

Modify the file `controller.ino` as follows:

```
 1 void setup()
 2 {
 3     Serial.begin(9600);
 4 }
 5
 6 int valueA0;
 7
 8 void loop()
 9 {
10     Serial.print("X ");
11     Serial.println(valueA0);
12     delay(100);
13     valueA0 += 1;
14 }
```

Explanation of the changes

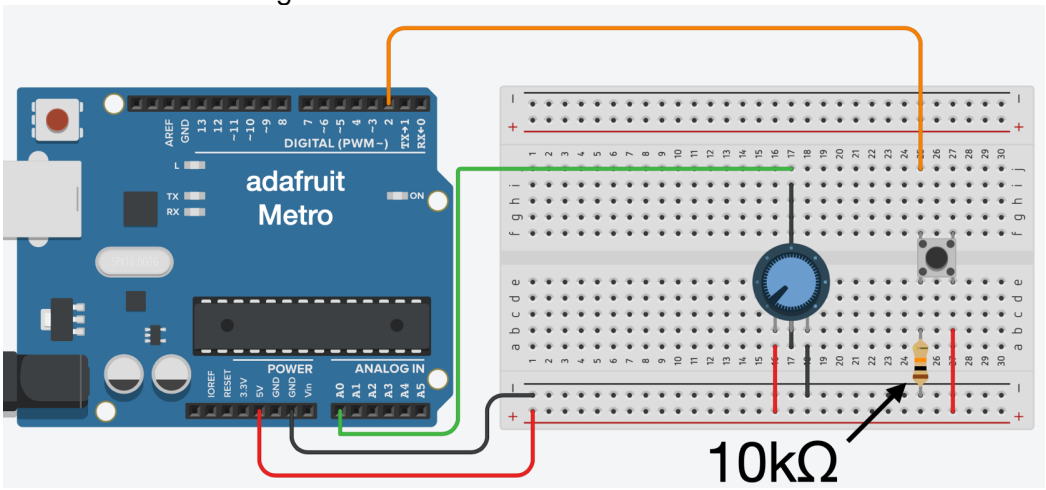
- In the function `setup()`, only the serial interface is initialized now.
- In line 6, a global variable `valueA0` of type `int` is declared. Since it is defined outside a function, it is a global variable. In C/C++, global variables without an initial value automatically get the value `0`.
- In line 10, the letter `'X'` followed by a space is output. This space will be important later.
- In line 11, the current value of the variable `valueA0` is output, followed by a line break via `println`. This line break will also play a role later.
- In line 13, `valueA0` is incremented by one in each pass of the `loop()` function.

Task: Compile the modified program, flash the microcontroller, and compare the output in the serial monitor with the expected output.
Finally: `git commit -a -m "Step 3"` and `git push`

© Michael Lehn | CC BY-SA 4.0

Step 4: Tinkering and even more playing around

Grab a **breadboard, jumper wires, wires, a 10 kΩ resistor, a push button, and a potentiometer**, and build the following circuit:



Important:
- The middle leg of the potentiometer is connected to pin A0.
- One leg of the push button is connected to pin 2 (the other one via the resistor to GND).

Edit the file `controller.ino` as follows:

```
 1 void setup()
 2 {
 3     Serial.begin(9600);
 4     pinMode(2, INPUT);
 5 }
 6
 7 int valueA0;
 8
 9 void loop()
10 {
11     if (digitalRead(2)) {
12         Serial.println("A");
13     }
14
15     int analogValue = analogRead(A0);
16     if (valueA0 != analogValue) {
17         valueA0 = analogValue;
18         Serial.print("X ");
19         Serial.println(valueA0);
20     }
21     delay(100);
22 }
```

Task
Before the program is explained in detail, first observe for yourselves what it does:
- Compile the program, flash the microcontroller, and start the serial monitor.
- Observe what happens when you turn the potentiometer – especially at its extreme positions.

- Observe what happens when you press the push button.

Finally: `git commit -a -m "Step 4"` and `git push`

© Michael Lehn | CC BY-SA 4.0

Explanation of the changes
- In **line 4**, `pinMode(2, INPUT)` configures digital pin 2 as an input.
- In **lines 11 to 13**, you see an `if` statement. The condition checks the value returned by `digitalRead(2)`. This function returns `1` (corresponding to **HIGH** or `true`) if a voltage of around 5 V is present at pin 2, and `0` (corresponding to **LOW** or `false`) if there is almost no voltage.
  If the return value is 1, the character `'A'` is sent via the serial interface in line 12 – including a line break via `Serial.println("A")`
- In **line 15**, a local variable `analogValue` of type `int` is declared and initialized with the return value of `analogRead(A0)`. Pin A0 is an analog input. A voltage between 0 V and 5 V is converted by the microcontroller into a digital value between `0` and `1023` (i.e., 10-bit resolution).

- The `if` **statement in lines 16 to 20** checks whether the new analog value `analogValue` differs from the previously stored value `valueA0`. Only if the value has changed,

  - is the new value stored in `valueA0` in **line 17**,
  - is an output generated via the serial interface in **lines 18–19** (in the format `X <value>`).

  If the value remains unchanged, no output is generated. This avoids unnecessary repetition.

What you should remember for today:
- With `pinMode(2, INPUT)`, a digital pin is configured as an input.
- `digitalRead(2)` returns `1` if a voltage is present at the pin, and `0` if not.
- With `analogRead(A0)`, a voltage from 0 V to 5 V is converted into a value from `0` to `1023`.

Step 5: Fine-tuning

The behavior of the microcontroller is to be refined:
- **When the push button is pressed,** an `'A'` is to be sent via the serial interface.
- **As long as the push button remains pressed,** no further character may be sent.
- **When the push button is released,** an `'a'` is to be sent.
- **As long as the push button is not pressed,** no further `'a'` may be sent.

In short: the characters `'A'` and `'a'` should only be sent on a change of state – not while the button is being held.

Task

1. Create a global variable `pressedA` of type `bool`. For it to be global, it must be declared **outside a function** – e.g., after line 7:
   ```
   bool pressedA;
   ```
   Since global variables in C/C++ are automatically initialized with the value `0`, `pressedA` is initially `false`.
2. Change the `if` statement in `loop()` to

   ```
   if (!pressedA && digitalRead(2)) {
       Serial.println("A");
       pressedA = true;
   }
   ```
   The `!` negates the logical value of `pressedA`, `&&` is the logical AND. With this condition, an `'A'` is only output if the push button was not pressed before.
   At the same time, we remember with `pressedA = true` that the push button is now pressed.
3. Compile the program, flash the microcontroller, and observe the output in the serial monitor. With this change, part of the desired behavior is already achieved. However, the output of `'a'` on release is still missing. With a second `if` statement and a bit of puzzling, you should be able to reach the goal completely.
4. **If everything works:** `git commit -a -m "Step 5"` and `git push`

© Michael Lehn | CC BY-SA 4.0

Step 6: From the hello world program to the serial interface

In both terminals, change into the directory `python`. This directory is located next to the current directory `controller` – you get there with:
`cd ..`
`cd python`
or in one go:
`cd ../python`

There you will find the file `serial-reader.py` with the following content:

```
print("Hello World")
```

This is still a classic hello world program. Run it with:

`python serial-reader.py`

The output should – not very surprisingly – simply be: `Hello World` (with a line break).

Now we dare to take a giant step forward: we modify the Python program so that it reads the serial interface!

Along the way, we also learn (learning by doing) a few basic language features of Python.

Modify the program `serial-reader.py` as follows:

```
 1 import serial
 2 import time
 3
 4 ser = serial.Serial("/dev/ttyUSB0", 9600)
 5
 6 print("Waiting for serial input...")
 7
 8 while True:
 9     while ser.in_waiting > 0:
10         ch = ser.read(1)
11         print(ch)
12     time.sleep(0.1)
```

Indentation is not negotiable

When it comes to wrong indentation, Python is **just as tolerant as I am** – that is, **not at all**.

Unlike many other programming languages, Python has **no curly braces** to delimit loop or function blocks. Instead, the Python interpreter – just like the programmer – recognizes **solely from the indentation** which statements belong to which block.
In our example:

- The outer `while True:` loop starts in **line 8** and comprises **lines 9 to 12**.
- The inner loop `while ser.in_waiting > 0:` starts in **line 9** and comprises only **lines 10 and 11**.
- `time.sleep(0.1)` no longer belongs to the inner loop – you can tell this solely from the fact that it is indented at the same level as the inner `while`.

© Michael Lehn | CC BY-SA 4.0

Task

Run the program with `serial-reader.py`

You should see output that looks roughly like this (if necessary, turn the potentiometer or press the push button):

```
b'X'
b' '
b'4'
b'0'
b'4'
b'\r'
b'\n'
```

Exit the program with `Control-C`.
Update your repository with `git commit -a -m "Step 6"` and `git push`

Explanation of the output

- The program outputs one received byte per line.
- The data comes directly from the serial interface of the microcontroller.
- In Python, such bytes are represented as so-called byte objects – you can recognize them by the prefix `b'...'`.
- Unlike normal character strings (strings), a byte object here contains exactly one byte, for example:
  - `b'X'` stands for the byte with the ASCII value `0x58`, i.e., the letter `X`
  - `b' '` is the space character (`0x20`)
  - `b'\r'` is the control character carriage return (`0x0D`)
  - `b'\n'` is the line feed (`0x0A`)

  These last two characters are produced when `Serial.println(...)` is used in the Arduino program – because `println` automatically sends `\r\n` at the end.

Technical background

The variable `ser`, which is created by `serial.Serial(...)`, represents a special data structure that manages access to the serial interface.
An internal **buffer** is also created, in which received bytes are stored temporarily.

- Whenever a byte is received via the serial interface, it first ends up in this buffer.

- The variable `ser.in_waiting` always contains the number of bytes currently stored in the buffer.
- With `ser.read(1)`, you can read **the oldest byte** from the buffer.

This buffer works according to the principle of a **FIFO data structure** *(First In, First Out)*:
The byte received first is also the first one to be read.

What you should remember for today:

- With `import serial`, the module pyserial is included – it must be installed.
- `ser = serial.Serial(...)` opens the serial interface (here: `/dev/ttyUSB0`) with the specified baud rate.
  All further communication with the interface takes place via the variable `ser`.
- `ser.read(1)` reads one byte from the interface and returns it as a byte object.
- The condition `ser.in_waiting > 0` checks whether data has already been received.
- The prefix `b'...'` means in Python: this is a byte object, not a string.
- `import time` loads the built-in module `time` (it does not need to be installed separately).
- With `time.sleep(0.1)`, a pause of a tenth of a second is inserted – this relieves the processor and prevents unnecessary loop activity.

© Michael Lehn | CC BY-SA 4.0

Step 7: Homework!

You can program really great things with Python – you just need to get familiar with a few basics. To get you fit for this, here is a small homework assignment:
At the university, we have licenses for an interactive online tutorial:

👉 Online Python Bridging Course (OPB)

Task until the next session (in two weeks):
- Work through **chapters 1 to 11**
- Pass the **challenges "Chapters 1–7"** and **"Chapters 8–11"**

The chapters are **short and doable** – about 5–10 minutes each. So no panic!

Afterwards, you will have a good understanding of central language elements of Python, e.g.:
- **Variables** (chapter 4)
- **Numbers** (chapter 5)
- **Strings** (chapter 6)
- **Booleans** (chapter 7)
- **If statements** (chapter 8)
- **Lists** (chapter 9)
- **For loops** (chapter 10)
- **While loops** (chapter 11)

If reading the serial interface was not motivation enough for you – just wait for the next chill-out step 😉

🎮 Chill-out step: Breakout – almost done…

In the directory `python`, you will also find the Python program `breakout.py`. It is not yet the finished game – but exactly what you will be able to program yourselves next time, once you have diligently worked through the online Python course. 😉

But now: just try it out! Run the program: `python breakout.py`

But afterwards, we clean up
Remove `breakout.py` from your repository – because writing it yourselves is more fun!

```
git rm breakout.py
git commit -a -m "Done!"
git push
```

Next time, you will build the game yourselves step by step –
and understand exactly how Python talks to hardware.

© Michael Lehn | CC BY-SA 4.0